Depuración de programas 1
INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl

Iniciemos lubuntu

- Utilizaremos la distribución **lubuntu** en los laboratorios
- Para ingresar en un laboratorio tienen que utilizar el usuario **.\test** y la contraseña **test**
- Abriremos dentro de esta sesión la máquina virtual de oracle, Oracle VirtualBox y **iniciaremos la máquina virtual** seleccionada lubuntu

Ejercicios

- Dentro de linux, descarguen los ejercicios subidos en Aula de laboratorio, con el nombre de “Depuración 1” y descomprimanlos.

Pasos básicos para depurar

Hay muchas herramientas para depurar programas, es decir, maneras de buscar porqué nuestros resultados al ejecutar un programa no son correctos.

Pasos básicos para depurar

Esto se pueden dar por problemas lógico o de ejecución, per ¿Cómo podemos encontrarlos ?

Hoy veremos dos opciones, sin herramientas externas y utilizaremos las siguientes maneras:

printf → Ver que esta ocurriendo
assert → Validar condiciones

Ejercicio 1

De los ejercicios descargados, comencemos en VisualStudioCode y abramos la carpeta depuración 1, y por la terminal de VSCode CTRL+J e ingresemos a el Ejercicio 1 con cd.

```
estudiante@estudiante-virtualbox:~/Desktop/ClasesPA/Depuración 1$ ls
Ejercicio1  Ejercicio2  Ejercicio3  EjercicioExtras  Readme
estudiante@estudiante-virtualbox:~/Desktop/ClasesPA/Depuración 1$
```

CompileAndGo.sh

debug1.c

```c
#include <stdio.h>

int main(){
    int datos[5] = {10, 20, 30, 40, 50};
    int suma = 0;
    int promedio;

    for(int i = 0; i < 5; i++){
        suma += datos[i];
    }

    promedio = suma / 5;

    printf("Promedio = %d\n", promedio);
    return 0;
}
```

Ejercicio 1

Del Ejercicio1/debug1.c, responda las siguiente preguntas

¿Resultado esperado?

¿Resultado obtenido?

Agreguen un printf, para mirar que hace el programa, intenten que si printf sea lo más especifico para determinar correctamente que pasa.

Ejercicio 1

Agreguen un printf, para mirar que hace el programa, intenten que si printf sea lo más especifico para determinar correctamente que pasa.

Siendo específicos podemos ver que realmente esta pasando con nuestro programa

```
for(int i = 0; i < 5; i++){
    printf("i=%d, dato=%d, suma antes= %d\n", i, datos[i], suma);
    suma += datos[i];
    printf("suma despues=%d\n",suma);
}
```

Assert

Afirmación que nos permite determinar si alguna variable no cumple con algún rango que nosotros creemos necesario para correr nuestro programa

```
#include <assert.h>
...
assert(n > 0);
```

Verificamos que la variable n sea mayor que 0, si esto no se cumple el programa termina

Assert - Ejercicio2

Los asserts siempre se realizarán antes de hacer “algo peligroso”

Vamos a la carpeta Ejercicio2/assert1.c , ejecutelo con el bash y pruebe distintas entradas como por ejemplo

```
10
-4
```

¿Para qué podría ocuparse este assert?

Ejercicio3

Abran el archivo debug2.c y coloquen correctamente los printf y los asserts para corroborar que el programa corre con las variables esperadas y podamos seguir que corre internamente en este.

Pueden llamar los bash que existen para probar su programa con distintos inputs !

```
./CompileAndGo1.sh
./CompileAndGo2.sh
```

Control Recuperativo

Control no obligatorio para recuperar la entrega del ticket de salida de la clase pasada.

En Aula se activo una sección para el desarrollo y entrega de esta evaluación.

Depuración de programas 1
INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl

Ejercicios extra 1 y 2

Coloque correctamente los printf necesarios para entender como se van modificando las variables y los asserts en caso de que se necesite verificar las variables que se ocupan

Depuración de programas 1
INF20125L

Felipe Dumont Ortiz
Felipe.dumont@usm.cl